

SENTINEL-TPD: Reproducible Runtime Evaluation of Tool Poisoning, Tool Shadowing, and Metadata Mutation in MCP

Rajeshkumar Sampathrajan
Independent Researcher

Abstract—Tool metadata can carry instructions that compete with an agent’s task, and a registered definition can change before invocation. These concerns motivate runtime gates, but a returned metadata verdict does not establish semantic attack prevention or safe server behavior. We evaluate SENTINEL-TPD at a fixed source revision, retaining poisoning, shadowing, risk decisions, response cost and blocking as separate study questions. The implementation combines ten deterministic text rules with metadata fingerprints, logical quarantine and a hash-chained audit. Across 47 pinned source templates, the registry prevents local enqueue for 17/27 attack-labeled cases (62.96%), while retaining 20/20 benign enqueues. On 70 authored challenges the corresponding counts are 4/26 and 37/44. All three source shadowing templates are missed. Fifteen lifecycle traces separate covered mutation/quarantine behavior from omitted metadata fields. Two process sessions measure five workloads of 100–300 repeated local calls; session 1 registry median cost is 17.237–17.806 microseconds per call, including audit and receipt storage. The contribution is a source-grounded coverage and lifecycle study with pinned inputs, explicit benign controls, observed local enqueues and regenerable results. The results characterize an implemented metadata gate; model selection, hidden server behavior and process isolation require additional mechanisms and experiments.

Index Terms—Model Context Protocol, tool poisoning, tool shadowing, runtime metadata gates, deterministic detection, reproducibility

1. Model Context Protocol Security and Threat Landscape

Agent hosts expose external capabilities through tool names, descriptions and argument schemas. In Model Context Protocol (MCP), discovery and invocation are separate events: the host consumes a catalogue and later sends a tool call. Descriptions enter a model-facing context, while the host determines which registration and connection supply the selected tool [1], [2]. A malicious instruction embedded in a definition can influence subsequent reasoning; a changed definition or competing identity can undermine an earlier check. These are different boundaries, so a useful evaluation records which one its mechanism actually mediates.

This study examines SENTINEL-TPD, a deterministic tool-metadata gate with registration and call-time checks [3]. The fixed revision accepts plain Python dictionaries, scans selected text fields, records accepted metadata fingerprints, and rejects unknown, quarantined or

changed definitions at invocation. Its name reflects tool-poisoning detection, but its supplied rule for cross-tool steering is not a general detector of name squatting, capability conflict or typographical impersonation. We keep all of those patterns in the evaluation so the distinction is visible rather than dropping cases the system may miss.

The poisoning patterns motivating the study include unrelated prerequisites, fake enabling prerequisites, argument hijacking, XML and system-tag injection, parasitic toolchains, cross-tool exfiltration, message hijacking, Unicode obfuscation, response injection, forced re-execution and annotation deception. Shadowing includes name squatting, capability override and typosquatting. The pinned fuzzd corpus supplies 24 poisoning templates, three shadowing templates and 20 benign descriptions; its own poisoning category labels are more granular than this twelve-pattern summary [4], [5], [6]. These are authored test definitions, not independently demonstrated harmful agent sessions.

1.1. Research questions and contributions

We ask four questions. RQ1: which poisoning and shadowing fixtures produce a rule finding, a deny decision and prevention of a local enqueue, and which benign fixtures are denied? RQ2: how do these decisions behave at the original five workload sizes when unique templates are repeated? RQ3: what does the registration plus fingerprint lifecycle check add beyond registration-only or rescanning? RQ4: what are its local decision-time and Python-allocation costs under a stated measurement boundary?

The contribution is an implemented-system characterization and reproducible counterexample set. We preserve the attack taxonomy and architecture questions, but replace unspecified weighted risk functions with the rule actually executed. We separate unique cases from repetitions, warning signals from enforcement, conditional blocking from all-attack blocking, and metadata mutation from hidden execution behavior. This study does not claim invention of reference monitors, regex classifiers, fingerprints, quarantine or audit chaining.

2. Existing Security Approaches for Model Context Protocol

2.1. Direct MCP and agent-security studies

MCP-Scanner, MCPXkit and MCP-SafetyBench study security analysis, attack simulation and agent safety at

broader boundaries [7], [8], [9]. Threat modeling and MCP ecosystem systematization identify poisoned metadata and trust-boundary failures [10], [11]; MCPTox and automatic red teaming study tool poisoning with agent and server settings [12], [13]. Prompt-injection reviews provide attack taxonomies and defense context [14], [15]. Their evaluation units are not interchangeable with a local dictionary scan. We use their distinctions to specify our boundary rather than comparing headline percentages from different models, tasks or datasets.

Layered detection in MCP-Guard, metadata-guided execution analysis in MCPShield, and semantic tool attacks are particularly relevant [16], [17], [18]. Autonomous remediation studies a further intervention after an anomaly [19]. SENTINEL-TPD has neither learned semantic scoring nor attention-based remediation. Its explainable fixed rules and fingerprint checks offer a different operational cost and scope. We do not reproduce those systems here and therefore make no empirical superiority claim over them.

Identity and execution-time authority are also established concerns. ETDI and related capability and reference-runtime work bind definitions or authority to identities and current policy [20], [21], [22]. The evaluated Sentinel registry is indexed by raw tool name. A multiserver host must provide its own namespace and route binding. A separate dispatch-conformance artifact examines that separate host boundary; its authored-adaptor outcomes are not substituted for the present content-detection results [23].

2.2. Adjacent studies retained with explicit boundaries

The original study also drew on multimodal poisoning, Shadow AI governance, mutated web payloads, federated-learning poisoning, DNS parsing, container security and medical-image poisoning [24], [25], [26], [27], [28], [29], [30], [31], [32], [33]. These works preserve useful methodological context: changing representations can alter detector coverage; logging and visualization can expose failures; and input validation differs from execution isolation. Their domains and attack objects remain distinct. Shadow AI denotes unsanctioned AI use, while tool shadowing here concerns competing tool definitions. ShadowCast and federated-learning poisoning change training data or representations, rather than an MCP catalogue. Appendix B records these boundaries so related terminology does not become evidence of MCP defense effectiveness.

3. Runtime Threat Detection and Security Architecture

3.1. Threat model and supported observations

The attacker may supply or revise metadata text and schemas, including nested strings. A source case may additionally describe an agent-selection or tool-result attack; if that surface is absent from the metadata API, it is explicitly marked as a projection or out-of-scope observation. The trusted local host invokes the gate and enqueues only allowed calls to an inert recording sink.

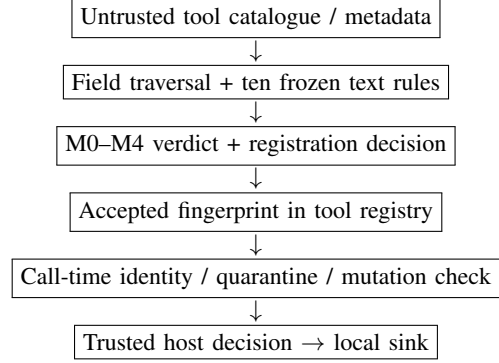


Figure 1. Implemented Sentinel architecture. Audit events accompany registration and invocation. The sink records an enqueue; it does not execute a remote tool.

The implementation, its rule configuration, the host’s enforcement decision and the sink are trusted. No adversary is allowed to modify Python code, the audit process or the measurement runner during a scored replay.

The protected local property is admission and invocation under the declared metadata policy. A missed fixture may reach the local sink, but this does not show that a language model followed its instruction or that a remote tool performed harmful work. Conversely, an unchanged harmful implementation behind a clean metadata definition is not observable to this gate. No live model, credentials, network tool effect or third-party target is exercised.

3.2. Metadata analysis and poisoning risk

The full input is a tool dictionary, but the fingerprint representation is a three-field projection:

$$F(M_i) = \{\text{name, description, inputSchema}\}. \quad (1)$$

The implementation also accepts an `input_schema` alias. Scanning visits string-valued top-level name and description, and recursively visits descriptions and titles inside the input schema. Top-level title and arbitrary annotations are not automatically traversed or fingerprinted. An input tree and a model’s actual displayed definition therefore need not have the same coverage.

Let $\mathcal{C}(M)$ be the strings traversed by the scanner; each signal j has a fixed regex p_j and severity s_j . The executed risk rule is

$$R_{\text{TPD}}(M) = \max(\{s_j : p_j \text{ matches } \mathcal{C}(M)\} \cup \{0\}). \quad (2)$$

The default threshold is $\tau = 3$:

$$\text{admit}(M) = \text{validName}(M) \wedge R_{\text{TPD}}(M) < \tau. \quad (3)$$

Invalid non-dictionary inputs and invalid names are denied. The scanner documentation also describes malformed-schema denial; our source-contract probe checks the implemented behavior rather than assuming that documentation proves validation. Multiple low-severity findings do not add into a higher risk score.

This operational definition replaces the earlier unspecified weighted poisoning, shadowing and behavior equations. It retains risk assessment as a study component, while identifying the actual M0–M4 mechanism. M1 urgency and M2 encoded-blob findings can remain allowed

TABLE 1. THE TEN FROZEN TEXT RULES. THE VERDICT IS MAXIMUM SEVERITY; M3 IS THE DEFAULT DENY THRESHOLD.

Signal ID	Level	Matched evidence (lexical scope)
TPD-IMPERATIVE-TAG	M4	Selected instruction, system, important, admin and hidden tags
TPD-DIRECT-MODEL-ADDRESS	M3	Selected direct instructions to the model or assistant
TPD-CONCEALMENT	M4	Selected instructions to hide activity from the user
TPD-PRIORITY-CLAIM	M3	Selected precedence, override and before-anything-else language
TPD-SENSITIVE-PATH	M3	Selected credential paths and secret-related terms
TPD-CROSS-TOOL-STEER	M3	Selected instructions about other tools or future messages/calls
TPD-PARAM-SMUGGLE	M3	Selected requests for full conversation, context or system prompts
TPD-EXFIL-DEST	M4	Selected transmission verbs near an explicit URL or email address
TPD-ENCODED-BLOB	M2	Long base64-like substrings; no decoding
TPD-URGENCY	M1	Selected urgent or threatening language

M1 and M2 alone remain allowed; neither finding counts nor weak-rule sums replace the threshold.

at the default threshold. Finding an indicator and denying its tool are therefore recorded separately.

3.3. Shadowing, identity and scope

The cross-tool steering rule matches instructions about other tools or future calls. It does not compare two tools’ edit distance or evaluate whether their claimed capabilities conflict. Names that resemble a trusted tool can be accepted when their metadata lacks a matched high-severity indicator. Conversely, an unknown name is denied at invocation because it was not registered, not because a similarity detector identified typosquatting. We preserve the three original shadowing families and test their actual observability through this API.

Source corpus records that embed executable setup or an agent-context attack require a projection before a plain dictionary scanner can consume them. Each projection retains its source file/ID and content hashes, and its applicability label. Projected descriptor decisions are reported separately from native metadata records. They do not establish execution of the original exploit.

3.4. Call-time monitoring, policy and quarantine

For an admitted definition, the registry stores

$$h_i = \text{SHA256}(\text{canonicalJSON}(F(M_i))). \quad (4)$$

Sorted keys make dictionary insertion order irrelevant. At invocation, the registry requires a valid name, a matching stored fingerprint and absence from quarantine. A covered mutation removes the registration and adds the name to quarantine. Restoring old bytes does not immediately clear that state. A clean catalogue refresh rescans definitions and can lift quarantine; direct registration and refresh are not identical recovery operations.

Logical quarantine prevents subsequent use through the enforcing registry. It is not an operating-system sandbox or process isolation. Call-time monitoring here means checking metadata and registration state at invocation hooks; it does not observe a running tool’s internal instructions, network actions or return values. The audit stores hash-linked events in memory, with optional file mirroring. Verification detects inconsistent retained links, but an actor who can rewrite an entire log can recompute a chain. The study makes no WORM or remote-attestation claim.

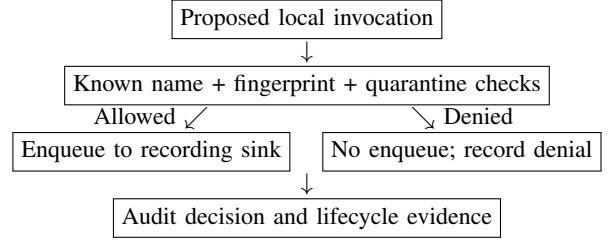


Figure 2. Decision flow. The allowed branch goes directly to its sink; the denied branch never joins the allowed execution path. Quarantine is registry state.

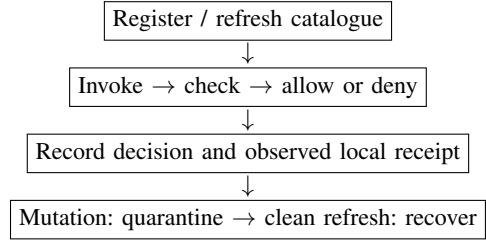


Figure 3. Implemented monitoring and response cycle. Recovery requires the stated refresh path; continuous remote behavior monitoring is outside this cycle.

4. Security Evaluation and Performance Analysis

4.1. Inputs, provenance and experimental units

The source corpus is pinned to fuzzd commit `f739e120cd7dc253fead2a1f13668f9535e4d8a5`. It comprises 24 poisoning IDs, three shadowing IDs and 20 benign entries. The benchmark repository’s MCPTox-derived definitions are a different artifact from the MCP-Tox paper’s agent-evaluation units [12], [34]. We do not pool either larger count into the present 47-case experiment.

The additional 70 authored challenge templates contain 23 poisoning, three shadowing and 44 benign controls. Source and challenge results are always separate. Each of the four arms sees every template once: 468 case/arm records per process session. Labels encode authored intent, not adjudicated semantic ground truth. TPA-022/023 tool-response payloads are projected into descriptions; TS-001/003 use declared authored name/peer context. These four are excluded from the native metadata-applicability subgroup. TPA-024 retains its annotations and adds declared authored destructive-description con-

TABLE 2. IMPLEMENTATION STATUS OF THE EIGHT ORIGINAL ARCHITECTURE MODULES.

Original module	Status	Supported boundary
MCP Client	External integration	A trusted host calls the library; no native MCP client or proxy transport.
Metadata Analysis	Selected fields	Text traversal and a canonical name/description/input-schema projection.
Poisoning Detection	Lexical rules	Ten regex signals with maximum severity; no learned classifier.
Shadowing Detection	Partial overlap	Cross-tool wording only; no provider identity, name similarity or capability model.
Runtime Monitor	Lifecycle checks	Pre-invocation fingerprint and quarantine state; no running-tool behavior observation.
Threat Assessment	Threshold policy	Maximum matched severity; original weighted scores are unimplemented.
Policy Enforcement	Host-mediated	Allow/deny and logical registry quarantine; no process isolation.
Security Log	Hash-chain audit	In-memory events, optional JSONL; no immutable storage or external anchoring.

TABLE 3. SCANNED, FINGERPRINTED AND HOST-CONSUMED FIELDS ARE DIFFERENT SETS.

Surface	Text scanning	Fingerprint	Possible host use
name, description	Yes, if strings	Yes	Discovery/selection text
inputSchema / alias	Nested description/title	Entire object	Input definition; call validation belongs to the host/server
Top-level title, annotations	No	No	May be displayed/consumed under the selected protocol version
Tool results, remote behavior	Not observed	Not observed	Outside the metadata invocation hook

LC-03/09 test covered mutations; LC-12/13 expose omitted top-level fields. A host must align its displayed/executed definition with its declared policy. Protocol fields vary by version; this is a coverage finding, not a new mechanism.

text, including within that subgroup. The 43-record subgroup therefore does not erase all augmentation or establish native annotation detection. Peer context is lineage, not an evaluated model-selection task.

The public artifact contains the authored corpus, source manifest, exact source hashes and pinned acquisition procedure. External descriptions are local-only: the pinned repository declares MIT in package/README metadata but supplies no LICENSE file or corpus-specific reuse terms. Public raw records keep case/tool hashes, rule IDs, field paths, match offsets and receipt evidence without payload excerpts. No corpus labels or thresholds are tuned after the scoring freeze.

The Sentinel source is pinned to `5ce5d56e57a0acd092435fca5d89b78d0b39ee7b`. All twelve vendored files retain their MIT notice and exact source identities. Rules and threshold are unchanged throughout scoring. The cases were visible during development and the authored challenges are specification-aware. There is no learned model, calibration fit, blind holdout or public preregistration. This study shares human authorship with Sentinel, so public code is not evidence of independent third-party validation.

4.2. Comparison arms and observed enqueues

An allow-all control exposes the workload denominator and sink behavior. A registration-only arm scans the initial definition and reuses admission without checking current metadata. The pinned Sentinel arm executes registration and call-time checks. A rescan-per-call arm applies the same scanner to each current definition but has no fingerprint or quarantine history. These are local host adapters around the same fixed rules, not reproductions of MCP-Guard, MCPShield or another published defense.

Each source and challenge template is evaluated once per arm with fresh state. The host then records whether it actually enqueued a fixed inert local request to the local sink. A denied fixture should have no receipt; a false-positive benign fixture also has no receipt, which is a

utility loss. Errors are retained as errors and never treated as successful prevention. The complete dataset remains separate from the per-kind repeated workloads.

4.3. Metrics and five workload levels

For attack-labeled cases, let N_a be the number tested, N_d those with any rule finding, N_b those denied with no receipt, and $N_{b,d}$ the blocked subset with a finding. We preserve detection and blocking as distinct metrics:

$$\text{SignalRate} = N_d/N_a, \quad (5)$$

$$\text{BlockingAll} = N_b/N_a, \quad (6)$$

$$\text{BlockingDetected} = N_{b,d}/N_d. \quad (7)$$

A zero denominator is reported as undefined. The last fraction is conditional on detection and cannot be interpreted as all-attack prevention. Confusion matrices count true/false positive and negative outcomes separately for any-rule findings and for denial. Benign enqueue completion is the fraction of benign-labeled cases with one expected local receipt; argument-schema validity and remote task completion are not tested. Attack labels express the fixture author’s intended attack, not a proven harmful effect in a real agent.

The original workload levels of 100, 150, 200, 250 and 300 calls are retained separately for poisoning, shadowing and benign source templates. Within a kind, a deterministic round-robin schedule repeats the same templates. We publish unique IDs and repetition counts for each row. A static detector does not learn from these repetitions; small percentage changes can arise from integer composition, not increasing capability with load. No repeated call becomes a new attack family.

4.4. Decision timing and allocation boundary

Two separate CPython 3.12.14 processes on one Intel Core i9-9880H (2.30 GHz), 16 GiB MacBookPro16,1 running macOS 26.6.2 produced the two full sessions. For each N , each process uses 10 warmup and 50 measured pairs with seeded randomized arm order. These

TABLE 4. CROSSWALK FROM THE SIX ORIGINAL METRICS TO MEASURED QUANTITIES (ORIGINAL TABLE NUMBERS IN PARENTHESES).

Original metric	Operational quantity	Interpretation
TPDR (III)	Rule-hit poisoning / poisoning labels	Report threshold denial and local prevention separately; not live-agent harm.
TSDR (IV)	Rule-hit shadowing / shadowing labels	Three unique source templates; no inferred identity-detector accuracy.
Response time (V)	Local pre-invocation decision interval	Mean/min/max for stated paths; no remote detection/containment clock.
ABR (VI)	Blocked / detected attacks	Conditional denominator; also report blocked/all and benign false blocks.
RSO (VII)	Paired $(T_S - T_A)/T_A$	Identical inert enqueues; baseline excludes a remote tool/model.
Memory cost (VII)	Incremental Python peak allocation	Separate traced bytes and baseline difference; not RSS or memory percent.

are within-process pairs, not independent process sessions. The monotonic `perf_counter_ns` clock uses `mach_absolute_time` (reported 1 ns resolution). Concurrency is one; CPU affinity, power and background load were not controlled or measured.

The paired assay repeatedly checks the admitted definition BENIGN-001 and enqueues an inert request with `{}` arguments. No argument-schema validation or remote task completion is asserted. Cold registration, copying, request serialization, collection and random ordering occur before timing. The timed loop includes the registry invocation check, native audit timestamps, hashing and list growth, plus receipt-dictionary storage. A and S receive the same request shape and N . Each time sample is the whole batch elapsed time divided by N ; medians and nearest-rank p95 are over 50 batch means.

A separate `tracemalloc` pass (one traceback frame) executes the same N checks and enqueues after setup and peak reset. It records each arm’s peak minus its pre-loop traced baseline, and the paired difference between arms. Audit and receipt accumulation therefore remain in allocation cost. Raw samples preserve nanoseconds and bytes. No confidence interval is inferred from dependent fixtures or repeated calls.

Response time here is the measured local decision interval. It is not elapsed time from a remote tool’s harmful behavior to containment, and blocked and allowed paths are summarized explicitly. The paired admitted-definition benchmark includes call-time registry verification, audit events and local receipt storage. Cold registration is outside that timer. Python allocation is measured separately using `tracemalloc`; neither peak allocation nor its incremental difference is process RSS or whole-system memory overhead.

4.5. Unique-case detection, prevention and benign controls

The three scanner-based arms agree on unchanged unique templates. Source prevention is 17/27: 17/24 poisoning and 0/3 shadowing. Challenge prevention is 4/26: 4/23 poisoning and 0/3 shadowing. The corresponding benign enqueues are 20/20 and 37/44; 7 challenge controls are false blocks. All allow-all cases produce receipts. In these scored fixtures every rule-hit attack is blocked, so the conditional blocked/hit ratio is 100%; its restricted denominator must not replace the all-attack rates. Allowed warnings are zero in these fixtures, although the M1/M2 policy permits them and dedicated unit controls exercise that distinction.

In the source native-metadata subgroup, 16/23 attack labels are blocked (69.57%) and 20/20 benign labels

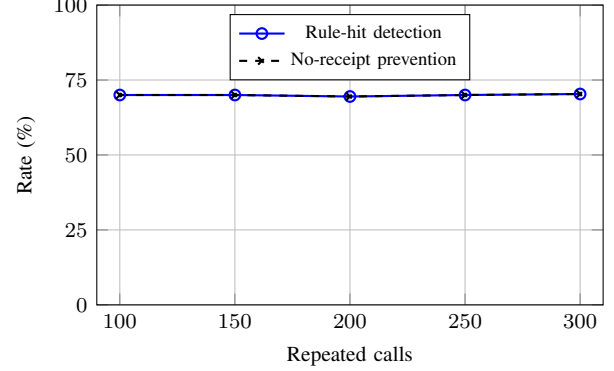


Figure 4. Poisoning rates on repeated source templates. The two curves coincide; the axis is fixed at 0–100%. Repetition changes integer composition, not a learned detector.

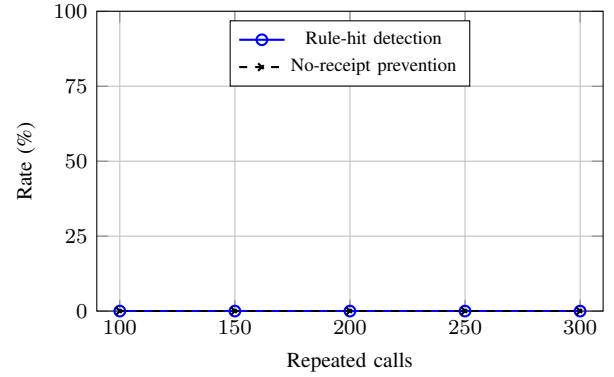


Figure 5. Shadowing rates on repeated source templates. The two curves coincide; the axis is fixed at 0–100%. Repetition changes integer composition, not a learned detector.

enqueue. The denominator is 23 attack and 20 benign records. The four omitted projections and TPA-024 augmentation are described above; this is a scoped subgroup, not agent-level attack success.

4.6. Repeated workload levels

Table 6 preserves the five workload levels for each kind. Every poisoning row repeats 24 unique IDs; shadowing repeats three and benign repeats 20. Figures 4 and 5 show count-derived rates on a common 0–100% axis. All benign workload calls enqueue. These streams assess this fixed repeated schedule and local decision cost; they do not estimate a learning or sensitivity trend.

TABLE 5. UNIQUE-TEMPLATE OUTCOMES. EVERY NUMERATOR COMES FROM A RULE FINDING OR OBSERVED LOCAL RECEIPT.

Split	Arm	Attack hit	Prevented	Blocked/hit	Benign enqueue	Warnings
source	A	0/27	0/27	–	20/20	0
source	R	17/27	17/27	17/17	20/20	0
source	S	17/27	17/27	17/17	20/20	0
source	C	17/27	17/27	17/17	20/20	0
challenge	A	0/26	0/26	–	44/44	0
challenge	R	4/26	4/26	4/4	37/44	0
challenge	S	4/26	4/26	4/4	37/44	0
challenge	C	4/26	4/26	4/4	37/44	0

A: allow all; R: registration only; S: actual Sentinel-TPD registry; C: rescan each call. Warnings are allowed cases with any rule hit. – means a zero denominator, not zero conditional effectiveness.

TABLE 6. REPEATED SOURCE WORKLOADS (SENTINEL-TPD, SESSION 1). DECISION INTERVALS IN MICROSECONDS.

Kind	N	U	H	B	E	Mean	Min	Max
poisoning	100	24	70	70	30	17.819	13.887	41.856
poisoning	150	24	105	105	45	18.054	14.025	62.798
poisoning	200	24	139	139	61	20.380	14.052	508.384
poisoning	250	24	175	175	75	17.787	13.786	61.182
poisoning	300	24	211	211	89	18.008	14.019	63.849
shadowing	100	3	0	0	100	23.130	22.084	26.014
shadowing	150	3	0	0	150	23.701	22.158	40.046
shadowing	200	3	0	0	200	23.972	22.089	68.493
shadowing	250	3	0	0	250	23.520	21.708	26.135
shadowing	300	3	0	0	300	23.371	22.275	27.634
benign	100	20	0	0	100	25.557	23.656	41.279
benign	150	20	0	0	150	25.720	23.453	77.633
benign	200	20	0	0	200	25.436	23.134	40.638
benign	250	20	0	0	250	26.175	23.366	98.025
benign	300	20	0	0	300	25.355	23.610	39.044

U : unique IDs; H : any-rule hits; B : no receipt; E : enqueues. Exact ID order and repetition multiplicities are in the frozen fixture manifest. Poisoning IDs TPA-001–024, shadowing TS-001–003, benign BENIGN-001–020 are repeated round-robin; all fifteen rows add no new independent templates.

TABLE 7. LIFECYCLE RECEIPT VECTORS; ONE ENTRY PER INVOCATION IN THE CASE’S ORDERED TRACE.

ID	Control or change	A	R	S	C	Expected	Expectation scope
LC-01	Unchanged control	[1]	[1]	[1]	[1]	[1]	Published lifecycle
LC-02	Dictionary key reordering	[1]	[1]	[1]	[1]	[1]	Published lifecycle
LC-03	Benign covered mutation	[1]	[1]	[0]	[1]	[0]	Published lifecycle
LC-04	Poisoned covered mutation	[1]	[1]	[0]	[0]	[0]	Published lifecycle
LC-05	Restore after quarantine	[1,1]	[1,1]	[0,0]	[0,1]	[0,0]	Published lifecycle
LC-06	Clean refresh recovery	[1,1]	[1,1]	[0,1]	[0,1]	[0,1]	Published lifecycle
LC-07	Poisoned refresh	[1]	[0]	[0]	[0]	[0]	Published lifecycle
LC-08	Catalogue withdrawal	[1]	[1]	[0]	[1]	[0]	Published lifecycle
LC-09	Nested schema mutation	[1]	[1]	[0]	[0]	[0]	Published lifecycle
LC-10	Unregistered name	[1]	[1]	[0]	[1]	[0]	Published lifecycle
LC-11	Changed name / typo	[1]	[1]	[0]	[1]	[0]	Published lifecycle
LC-12	Top-level title mutation	[1]	[1]	[1]	[1]	[0]	Host policy
LC-13	Annotation mutation	[1]	[1]	[1]	[1]	[0]	Host policy
LC-14	Null-schema admission	[1]	[1]	[1]	[1]	[0]	Doc probe
LC-15	Direct register after quarantine	[1,1]	[1,1]	[0,0]	[0,1]	[0,0]	Published lifecycle

1 is an actual local receipt, 0 is no receipt. LC-12/13 apply a stronger full-displayed-metadata host policy, not a promise of the pinned fingerprint contract. LC-14 probes the documented malformed-schema claim. These scoped discrepancies are not pooled into an upstream-vulnerability rate.

4.7. Lifecycle effects and field boundaries

The registry rejects benign covered mutation (LC-03) that rescanning accepts, showing that integrity checks add a different property from suspicious-text detection. Quarantine remains after byte restoration and direct registration (LC-05/15); a clean refresh recovers it (LC-06). Withdrawal and unknown/changed names are denied by the registry controls. A clean definition still passes after top-level title or annotation changes (LC-12/13), consistent with the field crosswalk. The null-schema probe (LC-14) also enqueues. Its documented validation expectation

and the stronger host policies are kept distinct from the published fingerprint behavior. No actual multiserver dispatcher binding or remote server behavior is tested.

4.8. Paired time and allocation cost

Across N , the registry’s median batch-per-call cost ranges from 17.237–17.806 μ s in session 1 and 17.554–18.269 μ s in session 2. The two sessions agree on all deterministic outcomes after excluding only timestamp-dependent audit-head hashes. Timing remains session-specific; the second session is not pooled to manufacture

TABLE 8. PAIRED INERT-ENQUEUE COST. TIME IS MICROSECONDS PER CALL; ALLOCATION IS KiB PER N -CALL BATCH.

N	A_{50}	S_{50}	S_{95}	Δ_{50}	Time %	A_K	S_K	Δ_K	$S_{50}^{(2)}$
100	0.199	17.237	20.634	17.034	8399.5	18.91	71.46	52.55	18.269
150	0.197	17.613	18.645	17.411	8870.0	28.39	106.10	77.71	17.554
200	0.185	17.601	22.142	17.414	9382.5	37.59	140.18	102.58	17.767
250	0.183	17.806	18.486	17.611	9637.3	47.11	174.88	127.77	18.155
300	0.175	17.251	18.018	17.076	9708.9	56.44	210.38	153.95	17.654

Session 1: medians (50), nearest-rank p95 (95), median paired time difference and overhead percent, median incremental Python peaks and their paired difference. Final column is session 2’s time median. Quantiles describe batch-per-call averages, not single-call tails. Time percent uses the very small allow-all baseline; no memory percentage or RSS is reported.

more independent hosts. Large time percentages reflect comparison with a sub-microsecond inert allow-all loop, not total agent latency. Python peaks include retained audit and receipt data and grow with the batch; they are not a leak diagnosis or resident-memory claim.

4.9. Interpretation and preservation of the original studies

The study retains each original question: poisoning detection, shadowing detection, response time, conditional attack blocking, runtime overhead and memory cost. Their operational definitions now match the code. The generated tables expose both source and challenge false positives, complete counts and workload composition. The figures use a single percentage scale for rates and keep counts in tables. Raw timing and allocation records are retained for reanalysis rather than reconstructing measurements from rounded cells.

Metadata mutation results explain why call-time checks and rescanning are not interchangeable. Fingerprints capture changes in the declared projection; rescanning may accept a clean but changed definition, while quarantine preserves mutation history until explicit refresh. Neither mechanism extends automatically to fields outside its projection or an unchanged remote implementation. Those boundaries are reusable test obligations for an integrating host.

4.10. Limitations and threats to validity

The source set is small, particularly its three shadowing templates. Authored challenge variants expand coverage but do not create a sample of deployed attacks. Intent labels have not received independent human adjudication. Template projection can change the observable attack; subgroup results and source lineage expose that choice. Known fixed rules, shared authorship and specification-aware cases create selection bias. There is no estimate of malicious prevalence, cross-model attack success or population recall.

The local sink proves enqueue occurrence and absence within this harness. It cannot certify remote side effects, model instruction following or route correctness in an arbitrary MCP host. Runtime results use one host and are descriptive. Audit logging and receipt storage contribute to cost, so a percentage relative to a tiny allow-all baseline is not a universal production overhead. Finite lifecycle cases and ordered local calls do not prove all concurrent schedules.

4.11. Ethics, harm and data availability

All calls end at an inert local sink. No credentials, remote victims or live attack payload execution are involved. Source-case descriptions may contain adversarial instructions, but they are treated as data. Public evidence avoids redistributing external payload text where corpus reuse terms remain uncertain. Publishing a source-contract discrepancy without its preconditions can mischaracterize a library; we pair such observations with controls and their actual field boundary.

The versioned research artifact is available at <https://github.com/rajesamp/sentinel-tpd-research/releases/tag/v0.1.0>. Both frozen process sessions, exact workload composition and all raw time/allocation samples are retained. Python 3.11 or later is required. Source acquisition uses pinned downloads; an authored-only mode operates offline. The source corpus is not redistributed. Reproduction regenerates tables from checked raw records and separately checks deterministic outcome agreement.

5. Conclusion

SENTINEL-TPD offers an implemented, auditable registration and invocation policy for selected tool metadata. This study evaluates the original poisoning and shadowing patterns, local denial and benign completion, five repeated workload sizes, metadata lifecycle behavior and measured runtime costs. Its contribution is the recorded coverage and boundary evidence under a fixed source and input pin. Further work should investigate independent agent integrations, adjudicated and adaptive attack datasets, model selection, result inspection and actual isolation before making broader runtime-security claims.

Acknowledgments and AI Assistance

OpenAI Codex assisted with source retrieval and checking, implementation of the study harness, authored fixtures, analysis and drafting of the Abstract, Sections I–V, appendices and accompanying artifact. Exact API model snapshots were not recorded. The human author is responsible for reviewing the sources, code, results, originality and all submission declarations. No AI system is listed as an author and no independent human peer review is claimed.

References

- [1] Model Context Protocol, “Model Context Protocol: Tools”, 2025. Specification version 2025-11-25; accessed October 6, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification/2025-11-25/server/tools>.
- [2] Model Context Protocol, “Model Context Protocol: Tools”, 2026. Specification version 2026-07-28; accessed October 6, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification/2026-07-28/server/tools>.
- [3] Rajeshkumar Sampathrajan, “sentinel-tpd”, *Software repository*. Revision 5ce5d56e57a0acd092435fca5d89b78d0b39ee7b. Accessed October 6, 2026. [Online]. Available: <https://github.com/rajesamp/sentinel-tpd/tree/5ce5d56e57a0acd092435fca5d89b78d0b39ee7b>.
- [4] ksek87, “fuzzd”, *Software and data repository*. Revision f739e120cd7dc253fead2a1f13668f9535e4d8a5; accessed October 6, 2026. [Online]. Available: <https://github.com/ksek87/fuzzd/tree/f739e120cd7dc253fead2a1f13668f9535e4d8a5>.
- [5] ksek87, “fuzzd tool-poisoning corpus”, *Software and data repository*. Revision f739e120cd7dc253fead2a1f13668f9535e4d8a5; accessed October 6, 2026. [Online]. Available: https://github.com/ksek87/fuzzd/tree/f739e120cd7dc253fead2a1f13668f9535e4d8a5/corpus/tool_poisoning.
- [6] ksek87, “fuzzd tool-shadowing corpus”, *Software and data repository*. Revision f739e120cd7dc253fead2a1f13668f9535e4d8a5; accessed October 6, 2026. [Online]. Available: https://github.com/ksek87/fuzzd/tree/f739e120cd7dc253fead2a1f13668f9535e4d8a5/corpus/tool_shadowing.
- [7] Parya Abadeh, Martin Lochner, Taha Ansari, Fattane Zarrinkalam, “MCP-Scanner: Detecting Security Risks in Model Context Protocol Systems”, *Proceedings of the ACM/IEEE 7th International Workshop on Engineering and Cybersecurity of Critical Systems*, 2026. [Online]. Available: <https://cybersciencelab.com/publications/mcp-scanner-detecting-security-risks-in-model-context-protocol-systems/>.
- [8] Yongjian Guo, Puzhuo Liu, Wanlun Ma, Zehang Deng, Xiaogang Zhu, Peng Di, Xi Xiao, Sheng Wen, “MCPXKIT: The Unified Toolkit for Analyzing Model Context Protocol Security”, arXiv:2508.12538, 2026. Version v2, May 25, 2026; arXiv record reports acceptance in IEEE TDSC. DOI: 10.48550/arXiv.2508.12538. [Online]. Available: <https://arxiv.org/abs/2508.12538v2>.
- [9] Xuanjun Zong, Zhiqi Shen, Lei Wang, Yunshi Lan, Chao Yang, “MCP-SafetyBench: A Benchmark for Safety Evaluation of Large Language Models with Real-World MCP Servers”, *International Conference on Learning Representations*, 2026. [Online]. Available: <https://arxiv.org/abs/2512.15163v2>.
- [10] Charoes Huang, Xin Huang, Ngoc Phu Tran, Amin Milani Fard, “Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning”, arXiv:2603.22489, 2026. Preprint, March 23, 2026. DOI: 10.48550/arXiv.2603.22489. [Online]. Available: <https://arxiv.org/abs/2603.22489>.
- [11] Shiva Gaire, Srijan Gyawali, Saroj Mishra, Suman Niroula, Dilip Thakur, Umesh Yadav, “Systematization of Knowledge: Security and Safety in the Model Context Protocol Ecosystem”, arXiv:2512.08290, 2025. Preprint, December 9, 2025; related 2026 proceedings chapter listed by Springer. DOI: 10.48550/arXiv.2512.08290. [Online]. Available: <https://arxiv.org/abs/2512.08290>.
- [12] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, Xiangyang Li, “MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers”, *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 42, pp. 35811–35819, 2026. DOI: 10.1609/aaai.v40i42.40895. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/40895>.
- [13] Ping He, Changjiang Li, Binbin Zhao, Tianyu Du, Shouling Ji, “Automatic Red Teaming LLM-based Agents with Model Context Protocol Tools”, arXiv:2509.21011, 2025. Preprint, September 25, 2025; author-hosted manuscript reports TIFS acceptance in 2026. DOI: 10.48550/arXiv.2509.21011. [Online]. Available: <https://arxiv.org/abs/2509.21011>.
- [14] Saidakhor Gulyamov, Said Gulyamov, Andrey Rodionov, Rustam Khursanov, Kambariddin Mekhmonov, Djakhongir Babaev, Akmaljon Rakhimjonov, “Prompt Injection Attacks in Large Language Models and AI Agent Systems: A Comprehensive Review of Vulnerabilities, Attack Vectors, and Defense Mechanisms”, *Information*, vol. 17, no. 1, Art. no. 54, 2026. DOI: 10.3390/info17010054. [Online]. Available: <https://www.mdpi.com/2078-2489/17/1/54>.
- [15] Narek Maloyan, Dmitry Namiot, “Prompt Injection Attacks on Agentic Coding Assistants: A Systematic Analysis of Vulnerabilities in Skills, Tools, and Protocol Ecosystems”, *International Journal of Open Information Technologies*, vol. 14, no. 2, 2026. [Online]. Available: <https://injoit.org/index.php/j1/article/download/2423/2053>.
- [16] Wenpeng Xing, Zhonghao Qi, Yupeng Qin, Yilin Li, Caini Chang, Jiahui Yu, Changting Lin, Zhenzhen Xie, Meng Han, “MCP-Guard: A Multi-Stage Defense-in-Depth Framework for Securing Model Context Protocol in Agentic AI”, *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 4877–4889, 2026. DOI: 10.18653/v1/2026.findings-acl.240. [Online]. Available: <https://aclanthology.org/2026.findings-acl.240/>.
- [17] Zhenhong Zhou, Yuanhe Zhang, Hongwei Cai, Moayad Aloqaily, Ouns Bouachir, Linsey Pang, Prakhar Mehrotra, Kun Wang, Qingsong Wen, “MCPShield: A Security Cognition Layer for Adaptive Trust Calibration in Model Context Protocol Agents”, arXiv:2602.14281, 2026. Version v3, February 24, 2026; preprint. DOI: 10.48550/arXiv.2602.14281. [Online]. Available: <https://arxiv.org/abs/2602.14281v3>.
- [18] Saeid Jamshidi, Arghavan Moradi Dakhel, Kawser Wazed Nafi, Foutse Khomh, “Semantic Attacks on Tool-Augmented LLMs: Securing the Model Context Protocol Against Descriptor-Level Manipulation”, arXiv:2512.06556, 2026. Version v2, 2026-05-21; inspected preprint. DOI: 10.48550/arXiv.2512.06556. [Online]. Available: <https://arxiv.org/abs/2512.06556v2>.
- [19] Zhiqiang Wang, Guanquan Shi, Yanting Wang, Yichao Gao, Hongsen Lang, Yunhao Yao, Haohua Du, Xiang-Yang Li, “Beyond Detection: Autonomous Anomaly Remediation for MCP Against Tool Poisoning Attacks”, *Proceedings of the ACM Web Conference 2026*, pp. 2974–2982, 2026. DOI: 10.1145/3774904.3792400. [Online]. Available: <https://dl.acm.org/doi/10.1145/3774904.3792400>.
- [20] Manish Bhatt, Vineeth Sai Narajala, Idan Habler, “ETDI: Mitigating Tool Squatting and Rug Pull Attacks in Model Context Protocol (MCP) by using OAuth-Enhanced Tool Definitions and Policy-Based Access Control”, arXiv:2506.01333, 2025. Version v1, 2025-06-02; inspected preprint. DOI: 10.48550/arXiv.2506.01333. [Online]. Available: <https://arxiv.org/abs/2506.01333v1>.
- [21] Ting Liu, “From Tool Connection to Execution Control: Benchmarking Security Invariants in MCP-Style Agent Runtimes”, arXiv:2606.29073, 2026. Version v1, 2026-06-27; inspected preprint. DOI: 10.48550/arXiv.2606.29073. [Online]. Available: <https://arxiv.org/abs/2606.29073v1>.
- [22] Alfredo Metere, “Attested Tool-Server Admission: A Security Extension to the Model Context Protocol”, arXiv:2605.24248, 2026. Version v1, 2026-05-22; inspected preprint. DOI: 10.48550/arXiv.2605.24248. [Online]. Available: <https://arxiv.org/abs/2605.24248v1>.
- [23] Rajeshkumar Sampathrajan, “Sentinel Dispatch Conformance: Reproducible Lifecycle Experiments”, *Software artifact, version 0.1.0*, 2026. Revision e874731f83c31ac0718f70c3c228dd9077928f99; accessed October 6, 2026. [Online]. Available: <https://github.com/rajesamp/sentinel-dispatch-conformance/releases/tag/v0.1.0>.
- [24] Yuancheng Xu, Jiarui Yao, Manli Shu, Yanchao Sun, Zichu Wu, Ning Yu, Tom Goldstein, Furong Huang, “Shadowcast: Stealthy Data Poisoning Attacks Against Vision-Language Models”, *Advances in Neural Information Processing Systems*, vol. 37, 2024. DOI: 10.52202/079017-1841. [Online]. Available: https://papers.nips.cc/paper_files/paper/2024/hash/6a2e30664b9647f97d7b9275358d083c-Abstract-Conference.html.
- [25] Deepak Puthal, Amit Kumar Mishra, Saraju P. Mohanty, Antonella Longo, Chan Yeob Yeun, “Shadow AI: Cyber Security Implications, Opportunities and Challenges in the Unseen Frontier”, *SN Computer Science*, vol. 6, no. 5, Art. no. 405, 2025. DOI: 10.1007/s42979-025-03962-x. [Online]. Available: <https://research.aber.ac.uk/en/publications/shadow-ai-cyber-security-implications-opportunities-and-challenge/>.

- [26] D. Puthal, N. K. Ray, S. P. Mohanty, “Shadow AI and the Expanding Attack Surface: Risks, Threats, and Countermeasures”, *OITS International Conference on Information Technology (OCIT)*, 2025. [Online]. Available: https://sarajumohanty.github.io/Mohanty_Curriculum-Vitae.pdf.
- [27] Marc Schmitt, Pantelis Koutroumpis, “Cyber Shadows: Neutralizing Security Threats with AI and Targeted Policy Measures”, arXiv:2501.09025, 2025. Preprint; related IEEE TAI publication listed by the authors’ institution. DOI: 10.48550/arXiv.2501.09025. [Online]. Available: <https://arxiv.org/abs/2501.09025>.
- [28] Cong Wu, Jing Chen, Simeng Zhu, Wenqi Feng, Ruiying Du, Yang Xiang, “WAFBOOSTER: Automatic Boosting of WAF Security Against Mutated Malicious Payloads”, arXiv:2501.14008, 2025. Preprint, January 23, 2025; author list is for the inspected preprint version. DOI: 10.48550/arXiv.2501.14008. [Online]. Available: <https://arxiv.org/abs/2501.14008>.
- [29] Chamara Sandeepa, Bartłomiej Siniarski, Shen Wang, Madhusanka Liyanage, “SHERPA: Explainable Robust Algorithms for Privacy-Preserved Federated Learning in Future Networks to Defend Against Data Poisoning Attacks”, *IEEE Symposium on Security and Privacy (SP)*, 2024. [Online]. Available: https://robust-6g.eu/wp-content/uploads/2025/12/SHERPA_Explainable_Robust_Algorithms_for_Privacy-Preserved_Federated_Learning_in_Future_Networks_to_Defend_Against_Data_Poisoning_Attacks.pdf.
- [30] Xueqing Zhang, Junkai Zhang, Ka-Ho Chow, Juntao Chen, Ying Mao, Mohamed Rahouti, Xiang Li, Yuchen Liu, Wenqi Wei, “Visualizing the Shadows: Unveiling Data Poisoning Behaviors in Federated Learning”, arXiv:2405.16707, 2024. Demo paper preprint; related ICDCS 2024 demo listed by an author. DOI: 10.48550/arXiv.2405.16707. [Online]. Available: <https://arxiv.org/abs/2405.16707>.
- [31] Xiang Li, Wei Xu, Baojun Liu, Mingming Zhang, Zhou Li, Jia Zhang, Deliang Chang, Xiaofeng Zheng, Chuhan Wang, Jianjun Chen, Haixin Duan, Qi Li, “TuDoor Attack: Systematically Exploring and Exploiting Logic Vulnerabilities in DNS Response Preprocessing with Malformed Packets”, *IEEE Symposium on Security and Privacy (SP)*, 2024. [Online]. Available: <https://zhangmm.net/publication/oakland24-tudoor/>.
- [32] Md Sadun Haq, Thien Duc Nguyen, Ali Şaman Tosun, Franziska Vollmer, Turgay Korkmaz, Ahmad-Reza Sadeghi, “SoK: A Comprehensive Analysis and Evaluation of Docker Container Attack and Defense Mechanisms”, *IEEE Symposium on Security and Privacy (SP)*, 2024. DOI: 10.1109/SP54263.2024.00182. [Online]. Available: <https://oaklandsok.github.io/papers/haq2024.pdf>.
- [33] M. Shyamala Devi, S. Priya, “Defending Data Poisoning Attack through Watermarked Friendly Noise Luminosity Activated Dense Layered UNet for Classification of Lung Disease”, *IEEE Access*, vol. 12, 2024. [Online]. Available: <https://panimalar.ac.in/assets/pdf/cse/research/journal/CSE-Journal-24-25.pdf>.
- [34] zhiqiangwang4, “MCPTox-Benchmark”, *Software and data repository*. Accessed October 6, 2026; repository not used as an evaluated corpus. [Online]. Available: <https://github.com/zhiqiangwang4/MCPTox-Benchmark>.

Appendix A.

Original Pattern and Study Coverage

The original twelve-pattern summary is preserved. The source corpus’s more granular category labels and all three shadowing families remain in the recorded per-family results. Unsupported semantic or runtime claims are retained as explicit research questions and limitations rather than counted as implemented defenses.

Appendix B.

Adjacent Research Boundaries

Table 10 preserves the adjacent literature’s domain and limited methodological connection. It does not imply algorithm transfer or a reproduced head-to-head comparison.

Appendix C.

Original Formal Expressions: Unimplemented Proposal

The following four expressions preserve the original design proposal. None defines the risk computation evaluated in this paper. The first is a metadata abstraction; the remaining three propose weighted scores:

$$M_i = \{N_i, D_i, C_i, P_i, R_i\}, \quad (8)$$

$$R_p(i) = \sum_{j=1}^k w_j x_{ij}, \quad (9)$$

$$R_s(i) = \alpha S_n(i) + \beta S_c(i) + \gamma S_t(i), \quad (10)$$

$$R(i) = \lambda R_p(i) + (1 - \lambda) R_s(i) + \delta B_i. \quad (11)$$

Here N, D, C, P, R denote name, description, capabilities, parameters and relationships; x denotes proposed poisoning features; S_n, S_c, S_t denote proposed name, capability and contextual similarity; and B denotes proposed behavioral evidence. The pinned code has no distinct capability/relationship analysis, fitted feature weights, corresponding similarity functions or observed execution-behavior term. No coefficients are estimated in this study. The measured implementation instead uses maximum rule severity and the stated fingerprint/quarantine policy.

TABLE 9. ALL TWELVE ORIGINAL POISONING PATTERNS AND THREE SHADOWING PATTERNS ARE RETAINED.

Source ID	Original pattern	Implemented scope or explicit gap
TPA-013	Unrelated prerequisites	No prerequisite/dependency reasoner.
TPA-014	Fake enabling prerequisites	No validation of claimed necessity.
TPA-015	Argument hijacking	Lexical parameter-smuggling signal; no approved-argument binding.
TPA-016	XML-tag injection	Selected tag patterns; not arbitrary XML interpretation.
TPA-017	Fake system tags	Selected tag forms in traversed strings only.
TPA-018	Parasitic toolchains	Lexical steering; no toolchain graph or call-sequence monitor.
TPA-019	Cross-tool exfiltration	Lexical paths/destinations; no runtime data-flow tracking.
TPA-020	Message hijacking	Lexical instruction signals; no message-origin proof.
TPA-021	Unicode obfuscation	No Unicode normalization, confusable matching or deobfuscation.
TPA-022	Response injection	Response projected into description; no actual tool-result inspection.
TPA-023	Forced re-execution	Response projection; no loop or invocation-count monitor.
TPA-024	Annotation deception	Annotations omitted; authored destructive-description context disclosed.
TS-001	Name squatting	Raw name key is not provider authentication; authored peer context.
TS-002	Capability override	No semantic comparison of competing capabilities.
TS-003	Typosquatting	No name-similarity detector; authored alias/peer context.

TPA-001–004, 005–008 and 009–012 additionally retain explicit-trigger, implicit-background-trigger and persistent-instruction families. Thus the source has fifteen poisoning category labels, not twelve independent implemented detectors.

TABLE 10. RETAINED ADJACENT REFERENCES AND TRANSFER LIMITS.

Ref.	Domain	Limit
[24]	Multimodal poisoning / ShadowCast	Representation poisoning is not MCP name/descriptor shadowing.
[25]	Shadow AI governance	Unsanctioned AI adoption differs from competing tool identities.
[26]	Shadow AI risk/governance	Organizational controls do not validate this metadata detector.
[27]	AI security and policy	Governance context; no reproduced MCP defense mechanism.
[28]	Mutated web/WAF payloads	Mutation motivates controls; web filtering rates do not transfer.
[29]	Federated data-poisoning defense	Training-data robustness differs from inference-time tool metadata.
[30]	Federated poisoning visualization	Failure visualization is methodological context, not MCP efficacy.
[31]	DNS parser logic	Malformed-input reasoning differs from this scanner's contract.
[32]	Container attack/defense	Execution isolation is a separate boundary absent from registry quarantine.
[33]	Medical-image poisoning	Image-model defenses and metrics are not implemented here.